Tools For Data Science

The data scientist's job

Data scientist has three tasks:

- Choosing and preparing data
- Using scientific tools to extract high-level knowledge (exploration, modelling, analysis)
- Communicate the results

Two sides covered here:

- Methodology (experimental setup, modelling choice, data analysis), algorithmic choices
- Practical tools, implemented algorithms.

Data Preparation

Storing and preparing data

- Storing data:
  - Find the best tool for your data (see next part). Are they organized in tables with relationships between them? Is it big?
- Preparing data:
  - Check data format
  - Track inconsistent data: look at distributions of variables, track outliers (see next slide), cross-check data by different means if possible.
  - Data fusion:
    - cross field between database if needed,
    - choose if you can live with default values for missing fields (‘outer/left’ join) or if you need to remove data (‘inner’ join),
    - make sure that you do not lose too much data, understand the possible biases.

Preparing simulations

Prepare input data

- Select interesting fields.
- Isolate them from primary source (different files, different databases).
- Version the data! Reproductibility is key to science!
- Optimise input for big models: use different tables/files, find out which fields are used more heavily etc.

Develop and run model (ABM, Monte-Carlo)

- Estimate the computational complexity of the model.
- Find the right language for it.
  - Heavy load ➔ consider low-level language.
  - Light computation ➔ high-level language.
  - Agent-based models ➔ OO language, dedicated framework.
- Find the right (stable) libraries for your problem. Don't recode something which has been done unless there is a good reason. Stackoverflow & Github are your friends!
- Develop your own tools if needed (reusable functions etc).
- Prepare data flow, input and output. Prepare a parameter file, as easy to read as possible.

Develop and run model (ABM, Monte-Carlo)

Organise the simulations:

- Choose carefully the combination of parameters to test (part of your experimental setup…),
- If sampling continuous data, start with small samples.
- Do running time tests, extrapolate.
- Estimate the number of realisations for stochastic models to have significant results.
- Keep a log of the simulations.

When code stable, optimize code IF NEEDED, starting with low-hanging fruits (obvious bottlenecks).

Analysing data

Exploring variables independently

Start by exploring each variable independently:

- Compute typical behavior: mean, median.
- Compute dispersion: standard deviation, 25-75 percentiles.
- Plot distributions, try to understand why they look the way they look.
- Compute and understand outliers (using z-score for instance).

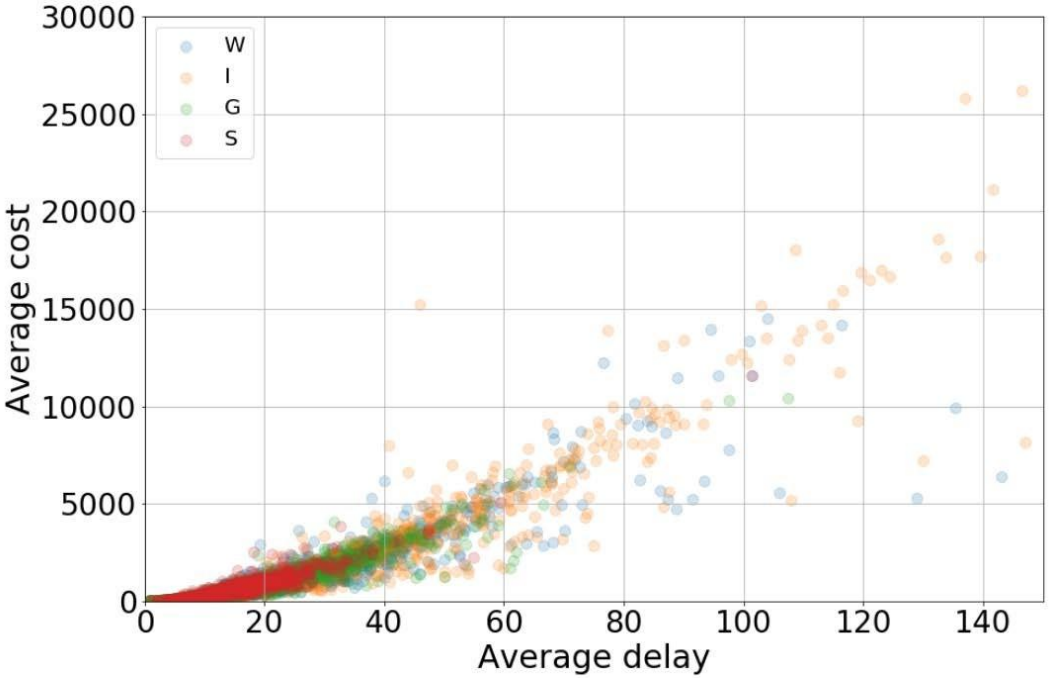
For dynamical data:

- Estimate stationarity (e.g. with moving mean and std), spot trends. Is the distribution at equilibrium?
- Compare ensemble mean to time mean.
- Consider detrending the data. Ex: daily traffic:
    - Compute ensemble mean at a given time.
    - Substract ensemble mean to each day.

Exploring relationships between variables

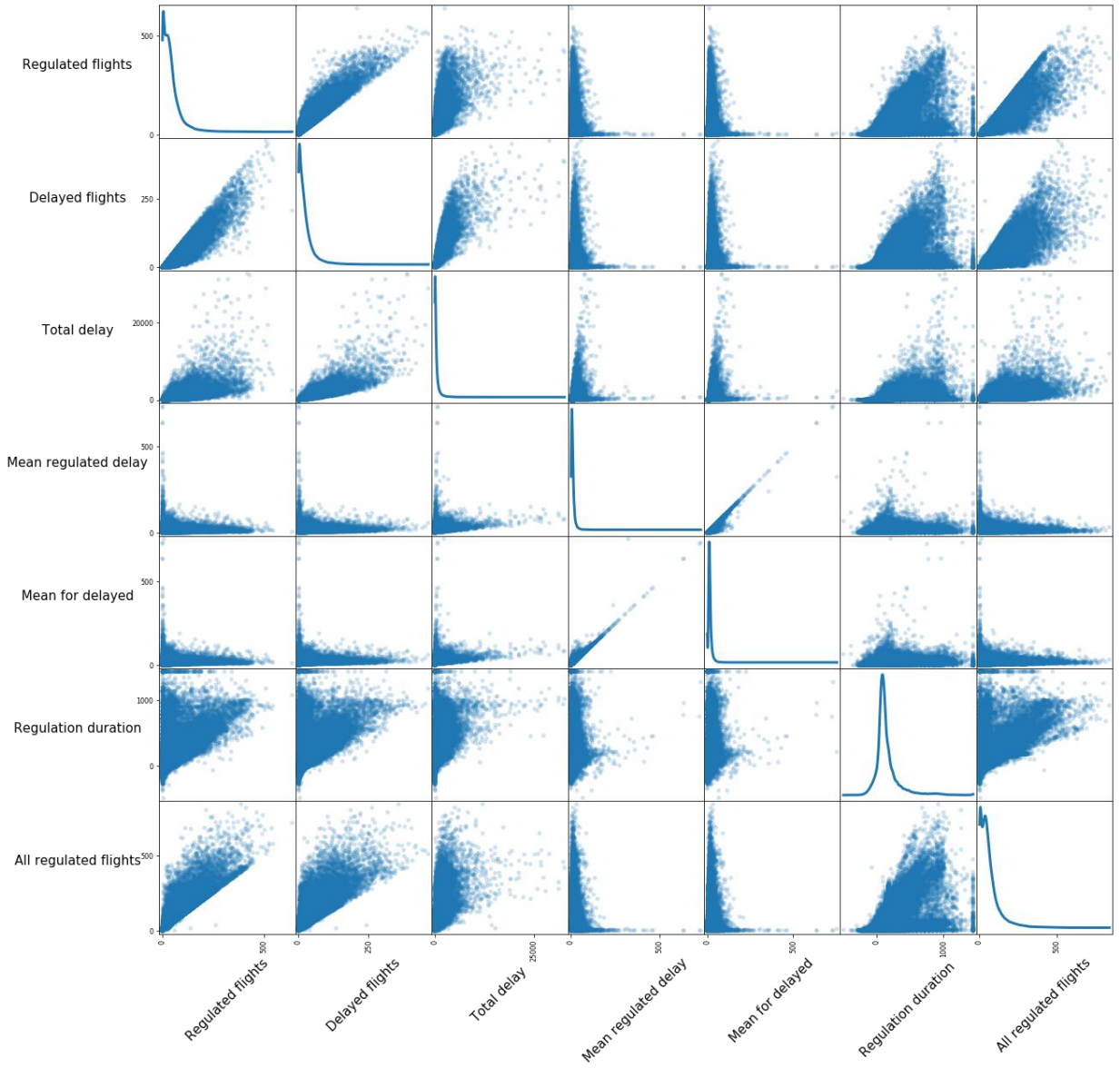
Understand the relationships between pairs of variables:

- Compute and analyse correlations: correlation matrix (+significance thresholds) etc.
- Use partial correlations coefficients on interesting variables.
- Do scatter plots between pairs of variables.
- Spot non-linear correlations with scatter plots.



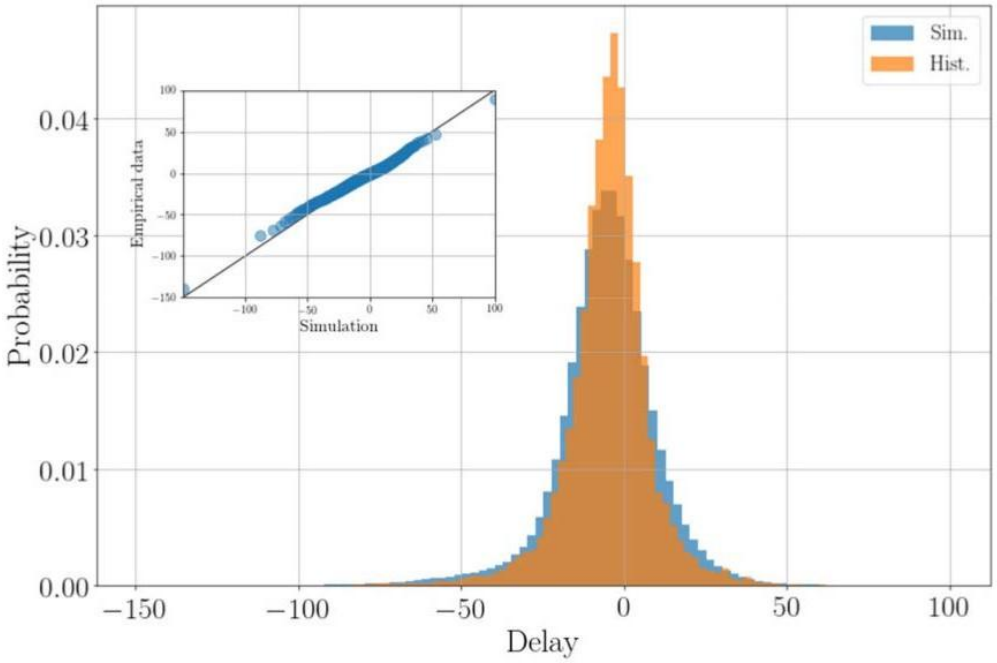
- Dyn.: consider computing partial corr., controlling for time

Exploring relationships between variables

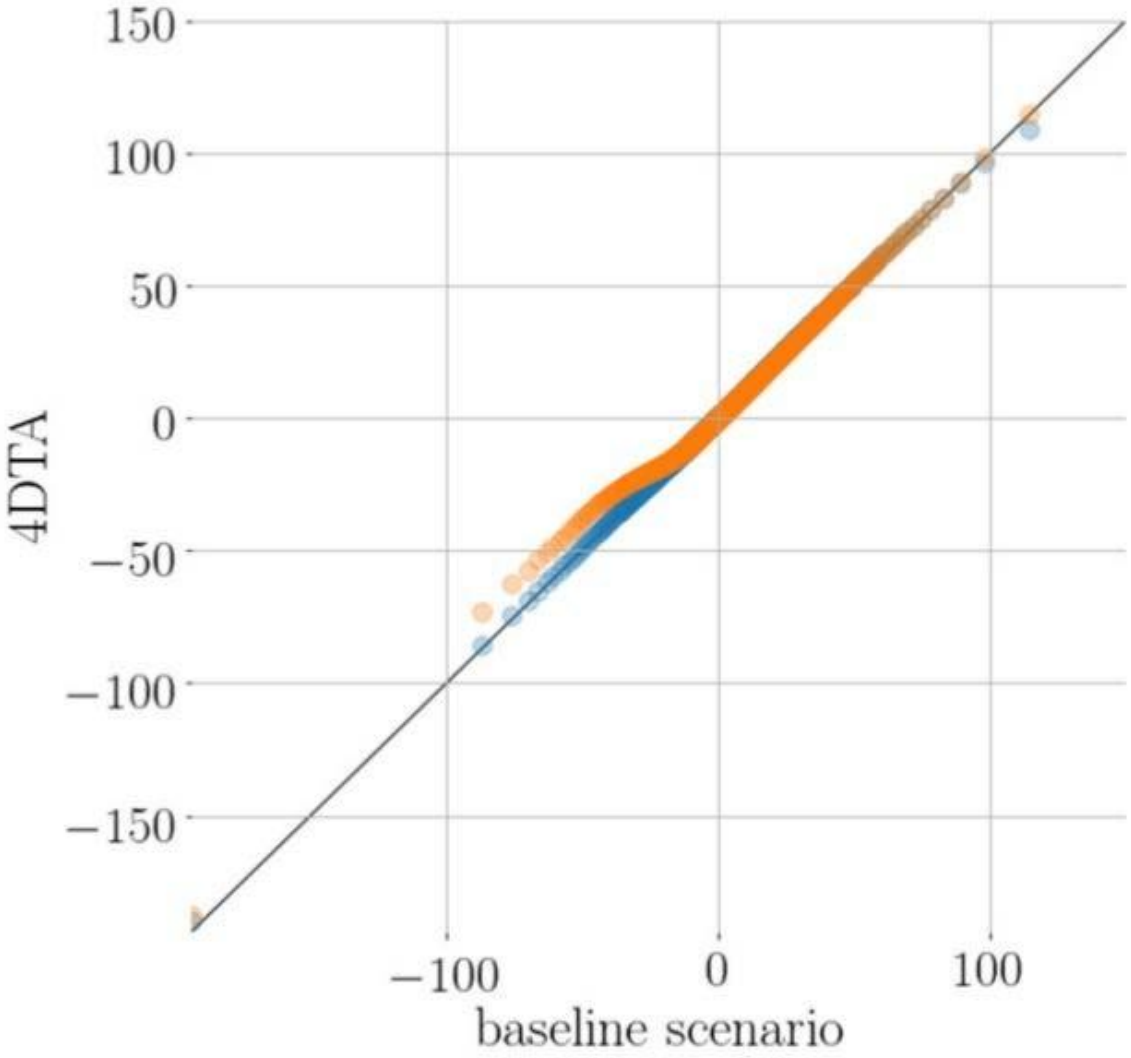


Analysing data (experimental results, empirical data)

- Standardise your data for analysis (remove mean, divide by std)
- For ordinal variables, use simple regressions on pairs of variables.
- Compare distributions between values of independent variables.
- Compute QQ plots



Analysing data (experimental results, empirical data)

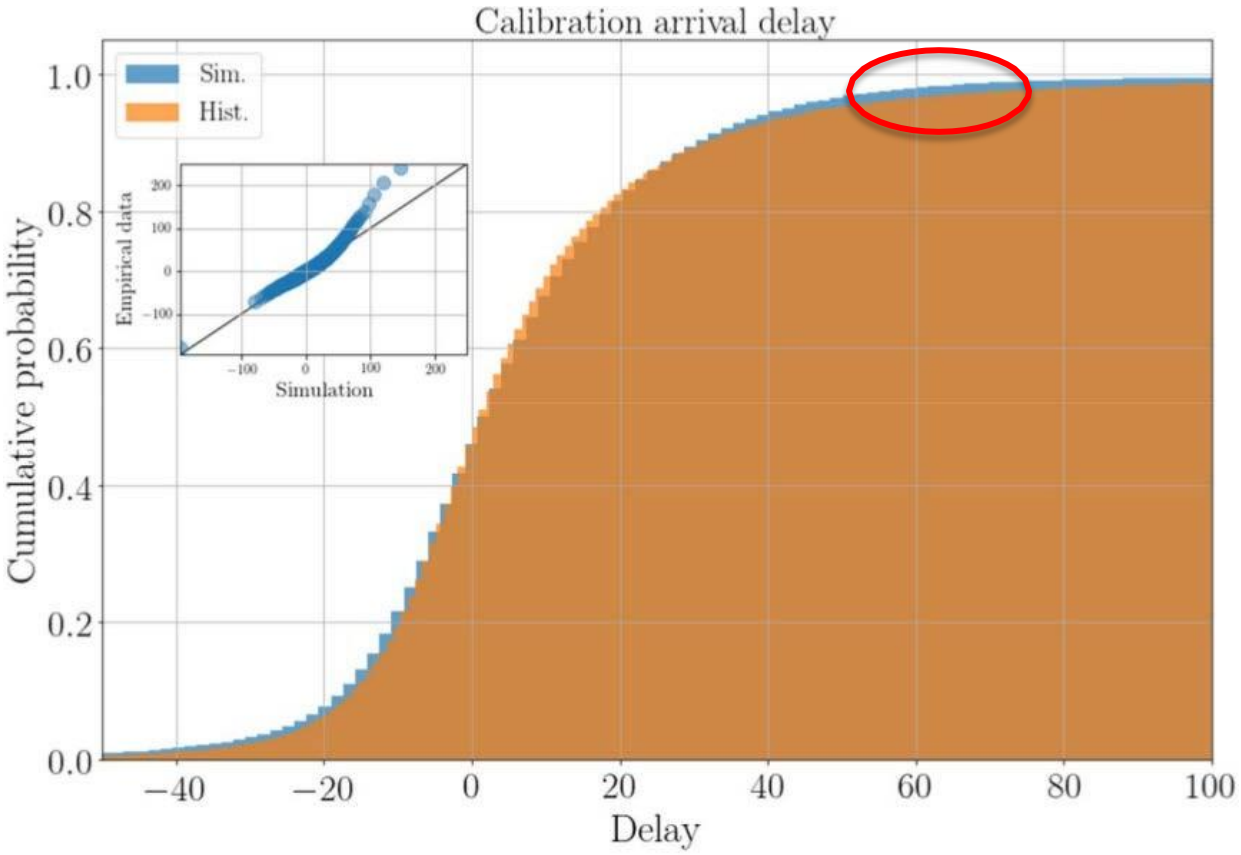


Analysing data (experimental results, empirical data)

- Depending on your hypotheses, find adequate statistical tests:
  - Exact test?
  - Bootstrapping, permutation?
  - Non-exact tests (chi2, F-test, etc…).
  - Decrease p-value for multiple-hypotheses tests (Bonferroni…)!
- Build confidence intervals (e.g. with bootstrapping)
- Don’t discard negative results!

Analysing data (experimental results, empirical data)

- Ex.: Kolmogorov-Smirnov test



Analysing data: data modelling

- Divide your data in two sets (randomly selected), e.g. 75%/25% for training and test data.
- Look at literature to find out which model would be relevant (part of your experimental plan…).
- Start with a simple model like ordinary least square. Include dummy variables for categorical variables to test their impact. Discard weak variables.

Out-sample R2: 0.8396442512711391

OLS Regression Results

Dep. Variable:	cost_delay	R-squared:	0.830
Model:	OLS	Adj. R-squared:	0.829
Method:	Least Squares	F-statistic:	7282.
Date:	Wed, 12 Jun 2019	Prob (F-statistic):	0.00
Time:	17:18:18	Log-Likelihood:	-9551.5
No. Observations:	17975	AIC:	1.913e+04
Df Residuals:	17962	BIC:	1.923e+04
Df Model:	12		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-0.0954	0.050	-1.924	0.054	-0.193	0.002
atfm_delay	0.9115	0.003	282.710	0.000	0.905	0.918
res_C	0.1304	0.050	2.615	0.009	0.033	0.228
res_G	0.0359	0.050	0.714	0.475	-0.063	0.134
res_I	0.2052	0.052	3.910	0.000	0.102	0.308
res_M	0.1752	0.054	3.249	0.001	0.070	0.281
res_O	0.2103	0.052	4.073	0.000	0.109	0.311
res_P	0.0986	0.051	1.925	0.054	-0.002	0.199
res_R	0.1002	0.054	1.841	0.066	-0.006	0.207
res_S	0.1199	0.051	2.359	0.018	0.020	0.219
res_T	0.0816	0.053	1.528	0.127	-0.023	0.186
res_V	0.0362	0.058	0.621	0.534	-0.078	0.151
res_W	0.0004	0.050	0.008	0.994	-0.098	0.099

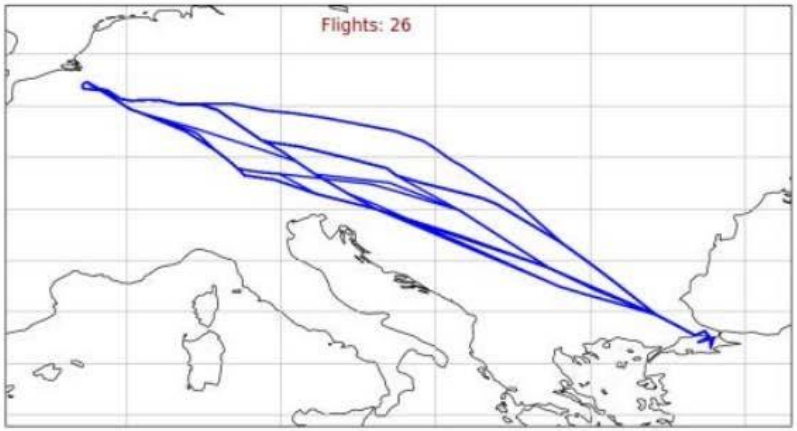
Analysing data: data modelling

- Train your chosen model, start with model as simple as possible (few leaves/neurons/parameters)
- Control for underfitting on training data (e.g. R) and overfitting on test data:
    - Compute "in-sample" score
    - Compute "out-sample" score.
- Compare with OLS. Play with parameters.
- Make prediction on new data. Check consistency, compare with other models if possible.
- Iterate and improve!

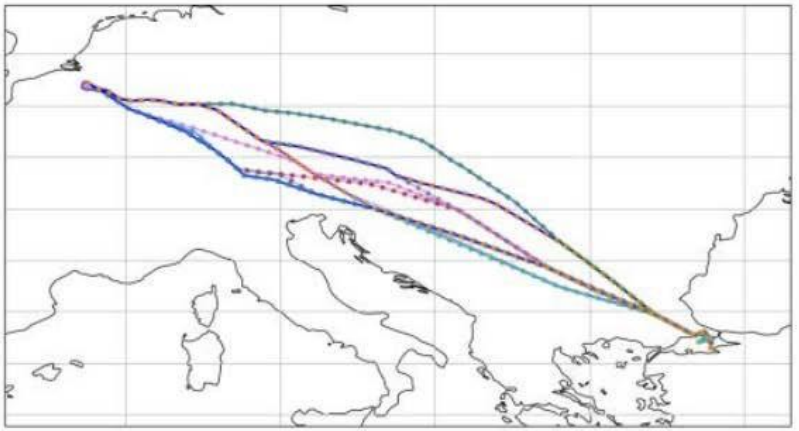
Analysing data: clustering

- Find the ‘right’ algorithm:
  - network data?
    - Yes ➔ community detection algorithms. Infomap, modularity, OSLOM.
    - No ➔ clustering algorithms. K-mean, mean-shift etc.
  - Fixed number of clusters (K-means)? Clusters can overlap (OSLOM)?
- Find the right “distance” function (when non-network). Start with something simple. Ex. with trajectories:
  - Differences in flight plan total distances.
  - Maximum lateral distances.
  - Etc…

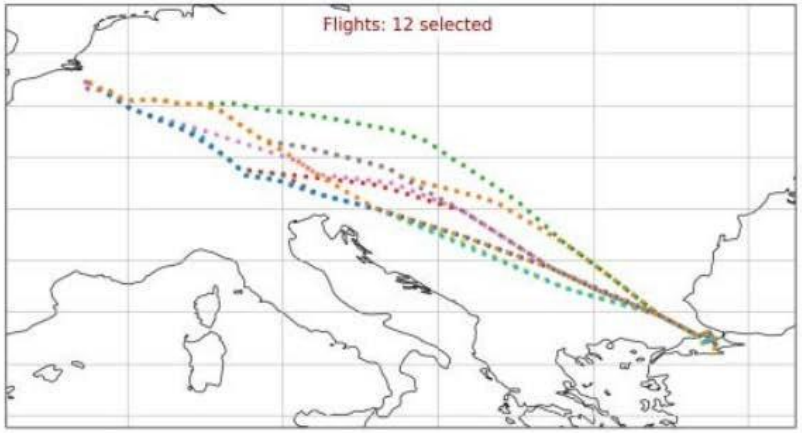
Analysing data: clustering



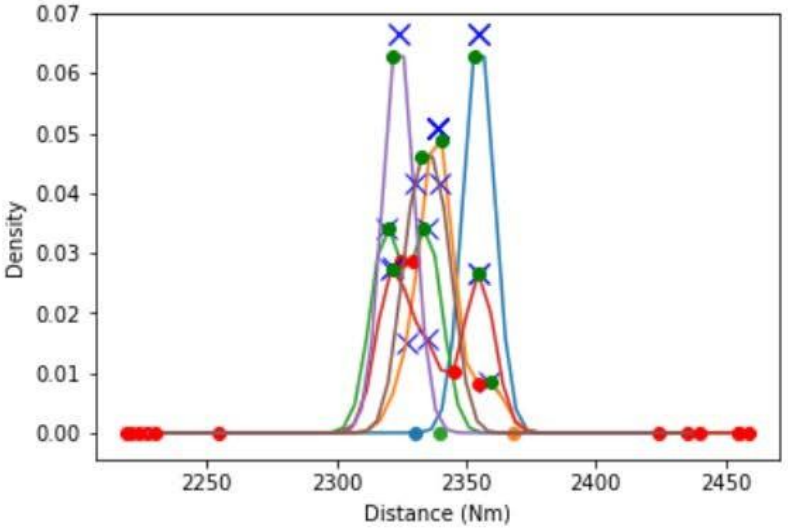
a) Original flights



b) Original and selected flights

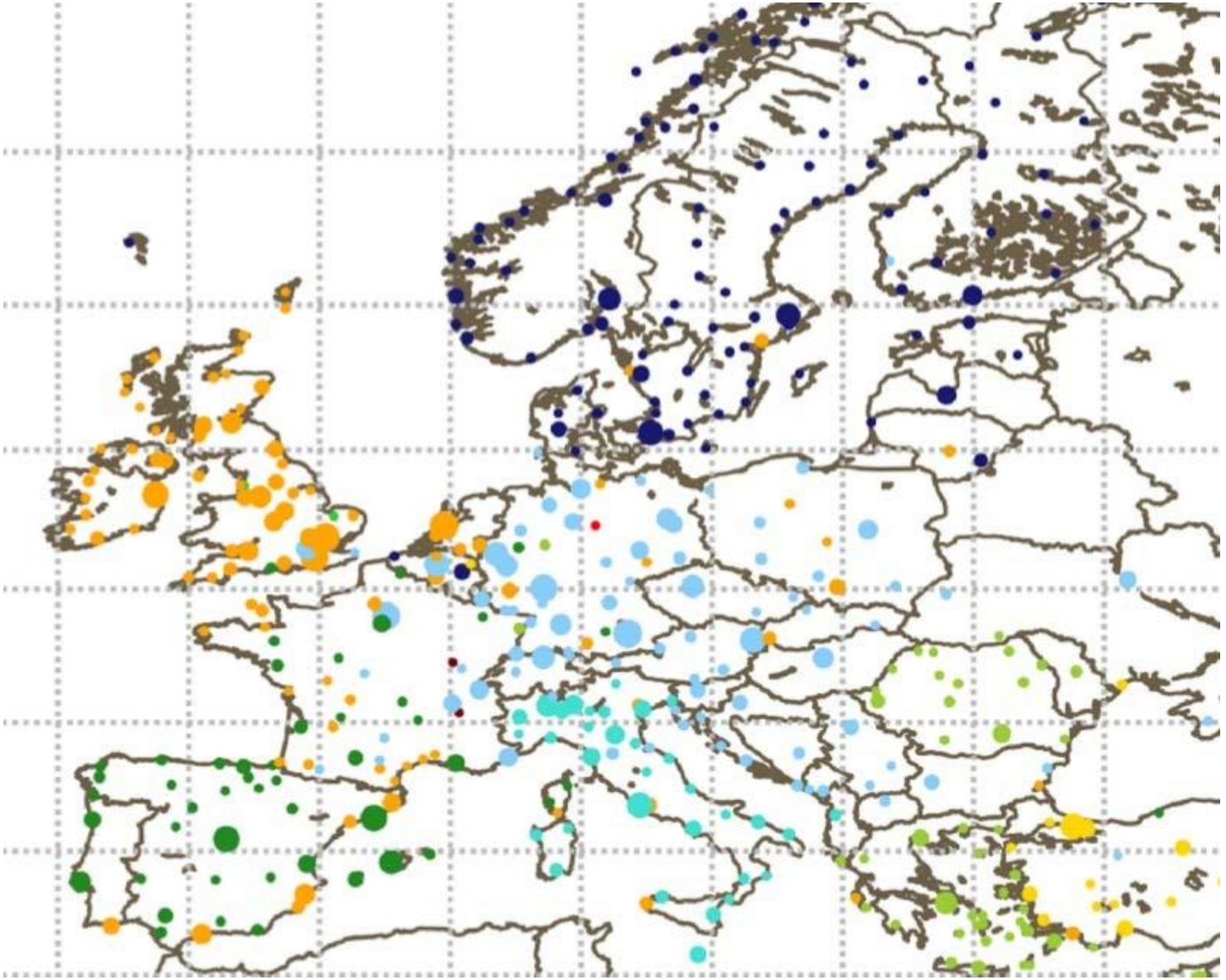


c) Selected routes



d) KDE clustering

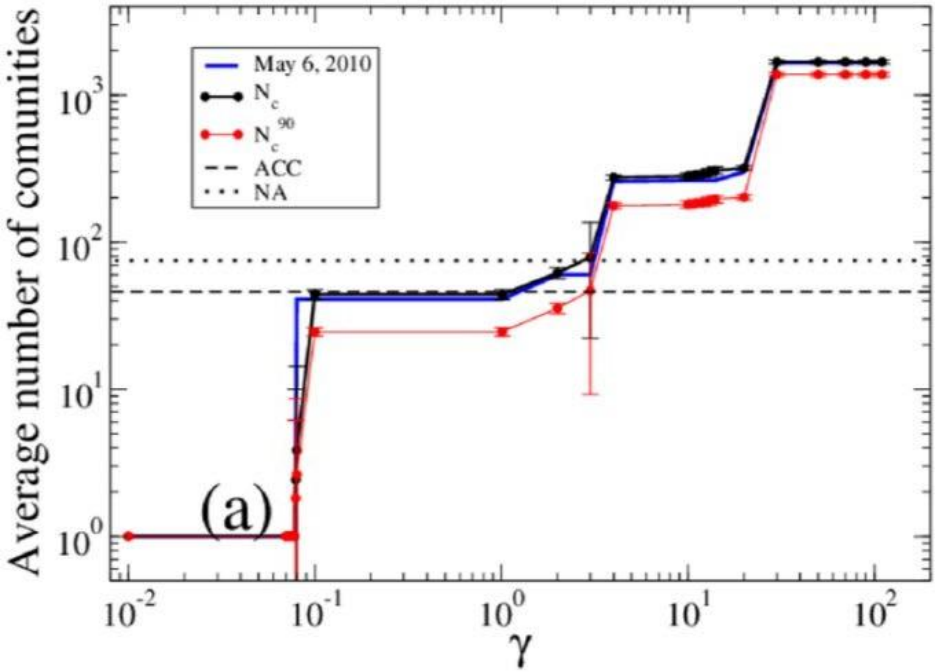
Analysing data: clustering



Analysing data: clustering

- Be aware of the scale: most algorithms have a natural, 'biased' scale.

(modularity based)



- Test robustness: modify input data, modify distance function.
- Use proper metrics to compare partitions: rand index, mutual information etc.

Show data

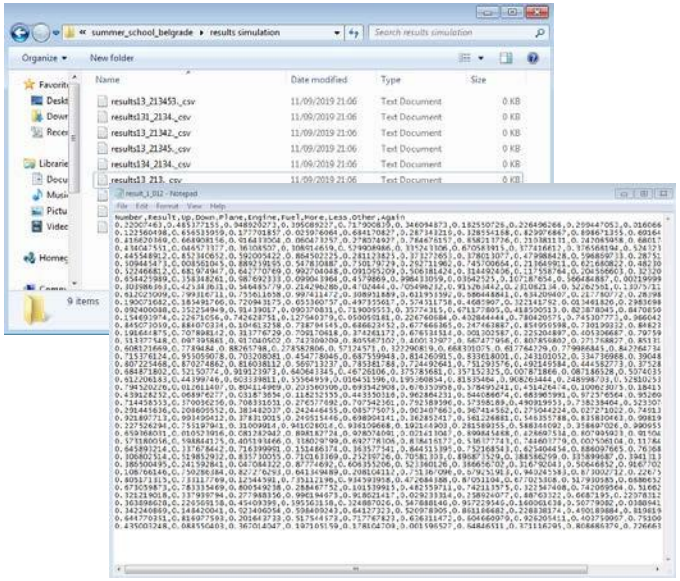
Three golden rules:

- target your audience,
- don’t mislead the audience,
- less is more (usually).

Tools for data science

Data storage

- File system
  - Advantage
    - Easy to use
    - Easy to output/input from models
    - Portable
    - Easy backup
    - CSV, etc.
  - Disadvantages
    - Hard to scale
    - Hard to process
    - Hard to join with other data
    - Difficult to maintain
    - Difficult for sharing/security
    - Hard to manipulate (change, update)



Data storage

- **File system**



- Linux file commands
  - cat
  - more
  - grep
  - sed
  - head
  - tail
  - split
  - pipe |
  - man
- awk
- open office (more rows)
- Sublime text/Atom (to check files)

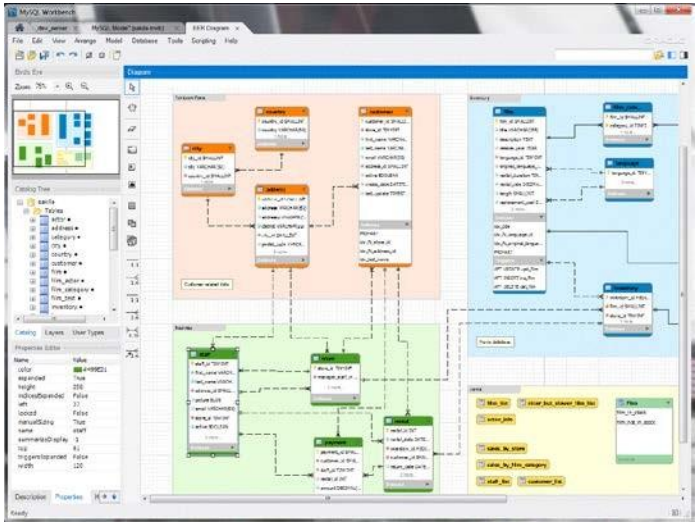




- Python / pandas might be useful (see next slides)
- Keep track of files in folder
- Use shell scripts (e.g. process all files in a folder)

Data storage

- Relational database
  - Advantage
    - Structured Query Language (SQL)
    - Easy to join data
    - Server based solutions
    - Secure access
    - View on data
    - Explore data with interface
    - Structure of database (UML)
    - API for programming languages
    - ACID principle (Atomicity, Consistency, Isolation, Durability)
  - Disadvantages
    - Might be difficult to scale
    - Slower access
    - Dynamic definition of data structure
    - Harder to keep traceability
    - Server configuration for perfomances



Data storage

- **Relational databases**



- MySQL
- MySQL Workbench
- Sequel Pro

- PostreSQL

- MariaDB

- SQLite



- Primary keys
- Indexes
- Numerical ids
- Foreign keys
- Configuration of server parameters
- Creation database structure

Data storage

- **Non-Relational database (NoSQL)**
  - **Advantage**
    - Address the scalability problem characteristic of SQL
    - Schema-free and built on distributed systems
    - Access time
    - Massive volumes of new, rapidly changing data types
    - Object-oriented programming
    - Dynamic schemas
    - API
  - **Disadvantages**
    - Relaxed ACID principles: eventual consistency (if there are no new updates for a particular data item for a certain period of time, eventually all accesses to it will return the last updated value)
    - It may lead to data loss (quality of application code)
    - Heterogeneity of systems (little uniformity among them)
    - Lack of standardisation (maturity)

Data storage

- **Non-Relational databases**



- Apache CASSANDRA
- Apache HBASE
- mongoDB
- Amazon Dynamo









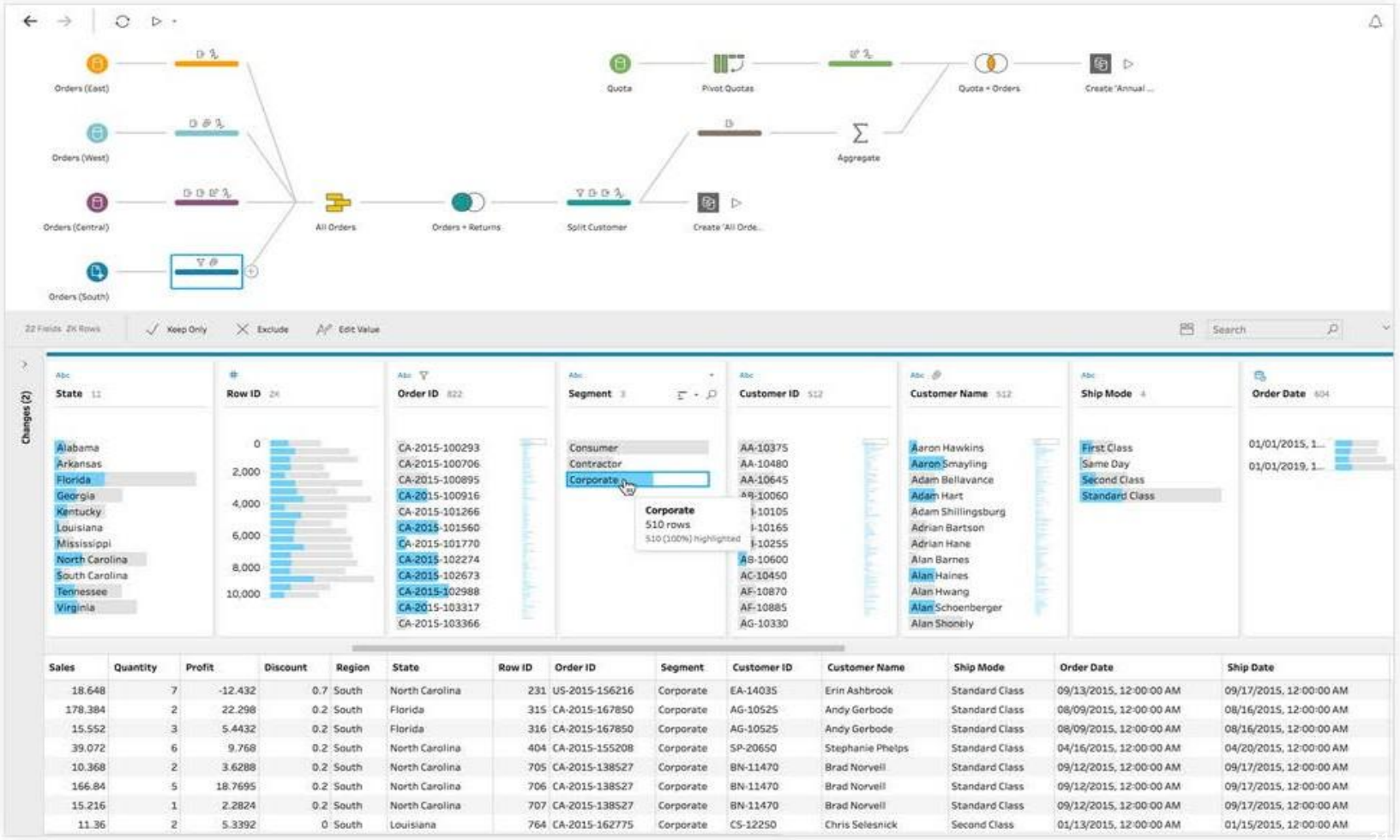




- Different implementations / alternatives
- Research online!

Data handling

- Tableau (commercial): data fusion, correction, cleaning, and analysis



Data handling

- Matlab (commercial, equivalent open-source: Sage, Scilab, Octave):
    - Excellent at numerical analysis. Lots of libraries.
- Excel! Open source Libreoffice can handle more rows.
- Python tools (free, open-source):
    - Jupyter notebook (interactive python shell).
        - Easy to use and easy to share the results.
        - Collaborative version from google.
    - Pandas: build “dataframes”.
    - Geographical data: cartopy, basemap, geopandas (built on pandas). Manage projections, distances etc.
    - Big community, open modules, easy to learn.

Data handling (jupyter)

e Drive

jupyter Empirical analysis (on 2014 data) Last Checkpoint: 08/02/2019 (autosaved)

Logout

File Edit View Insert Cell Kernel Widgets Help

Not Trusted | Python 3

Run Code

02:47:00 03:07:10 03:20:00

In [11]:

```
ff = lambda x: ''.join(x)[0]

mask = (df['cost_delay']>0.) & (df['atfm_delay']>0.)

dg = df.loc[mask, ['day', 'sid', 'atfm_delay', 'cost_delay', 'reason']].dropna().groupby(['day', 'sid']).agg({'atfm_
dg['atfm_delay2'] = dg['atfm_delay']**2
dg.dropna(inplace=True)
dg
```

Out[11]:

day	sid	atfm_delay	cost_delay	reason	atfm_delay2
2013-12-11	LLBGA11	8.500000	265.000000	G	72.250000
	E4N412A	6.222222	165.444444	I	38.716049
	E5EH12A	19.875000	853.718750	I	395.015625
	E5R512A	14.903226	546.064516	I	222.106139
	E5R512M	10.000000	270.000000	I	100.000000
	EDDG212M	7.375000	132.375000	S	54.390625
	EDDHA12M	31.580000	1545.040000	W	997.296400
	EDDLA12	46.523810	2926.857143	O	2164.464853
	EDDMA12M	13.875000	429.479167	W	192.515625
	EDDMA12N	22.931818	886.250000	W	525.868285
	EDDTA12	25.948718	1292.115385	W	673.335963
	EDDTA12A	67.864865	6654.567568	W	4605.639883
	EFHKA12	14.960000	495.240000	W	223.801600
	EGKKA12	23.942529	1103.034483	W	573.244682
	EGLCA12	43.413793	1845.344828	W	1884.757432
2013-12-12	EGLCA12A	11.000000	187.142857	W	121.000000
	EGLLA12	26.712230	1548.913669	W	713.543243
	EHFIR12L	8.864865	202.810811	W	78.585829

Model development

- Low-level languages for computational power: C/C++.
- High-level languages for ease of use:
    - Python:
        - Advantages: easy to learn and use, massive numerical libraries, full power of a 'real' language,
        - for simulations: simpy,
        - for ABM: Mesa,
        - for scientific tools (algebra etc): numpy, scipy,
        - for database(s) interface: alchemy
        - for file access (csv, excel, jason): pandas.
        - multiprocessing : simple CPU parallelisation.
- Numba: GPU parallelisation.

Rmk: on Windows, use Anaconda suite.

Model development

```
        total_delay_expected = np.round(total_delay_expected,2)
        num_zeros = np.count_nonzero(total_delay_expected==0)
        if num_zeros>1:

            if num_zeros==len(total_delay_expected):
                #print("AAAA",sibt,self.agent.FP.get_xit(),eibt_slots)
                incr = 0.00001
                min_index_non_zero = len(total_delay_expected)
            else:
                min_index_non_zero = np.amin(np.where(total_delay_expected>0))
                min_value_non_zero = total_delay_expected[min_index_non_zero]
                incr = (min_value_non_zero/2)/(num_zeros-1)

            for i in range(1,min_index_non_zero):
                total_delay_expected[i]=total_delay_expected[i-1]+incr

            total_delay_expected = np.round(total_delay_expected,5)
            #print(total_delay_expected)

        total_delay_expected = total_delay_expected.tolist()

        self.dict_costs_slots = {key: value for (key, value) in zip(slots_times, total_delay_expected)}
        self.dict_arrival_delay_slots = None
        self.cost_function_slots = None


    def request_time_propagate_delay(self):
        msg = Letter()
        msg['to'] = self.agent.aoc_info['aoc_uid'] #AOC
        msg['type'] = 'request_time_propagate_delay'
        msg['body'] = {'flight_uid':self.agent.uid}
        self.send(msg)
```

Model development

- Java:
  - Dedicated frameworks for ABMs (Java): Jade, Jadex.
  - Faster than Python.
  - Less obvious syntax, harder to learn.
  - Not so used for data handling and data visualisation.

- Don’t hesitate to use a version-control tool, like git. Quick to learn and very useful to keep track of changes, even if you develop alone. (rmk: free Github space for academics and students).
- Use bash scripts (or python scripts…) or equivalent to run batch of simulations.

Data analysis/modelling

- Python:
  - pandas/numpy: can quickly extract standard statistics.
  - scipy, scipy.stats: useful tools like z-scores.
  - scikit-learn: more in depth analysis. Dedicated to machine learning.
  - networkx: useful for network data. Can easily compute degree, strength, cliques etc.
  - TensorFlow (from Google), Keras (using TensorFlow and theano): deep learning (or not…).

Data analysis/modelling

- "R" (free, open-source):
    - very focused on statistics,
    - easy to learn and to use,
    - lots of libraries.
- Tableau, Matlab etc
- Julia: new language designed for data science. Still young, but promising.

Data analysis/clustering algorithms

- Python:
  - Clustering (Kmeans, mean shift, etc): scikit-learn
  - Community detection:
    - Infomap: now usable with Python (and networkx)
    - Simple modularity maximisation: include in networkx. More advanced capabilities in other non-standard library.
- OSLOM: powerful algorithm, tool written in C and executed with a script

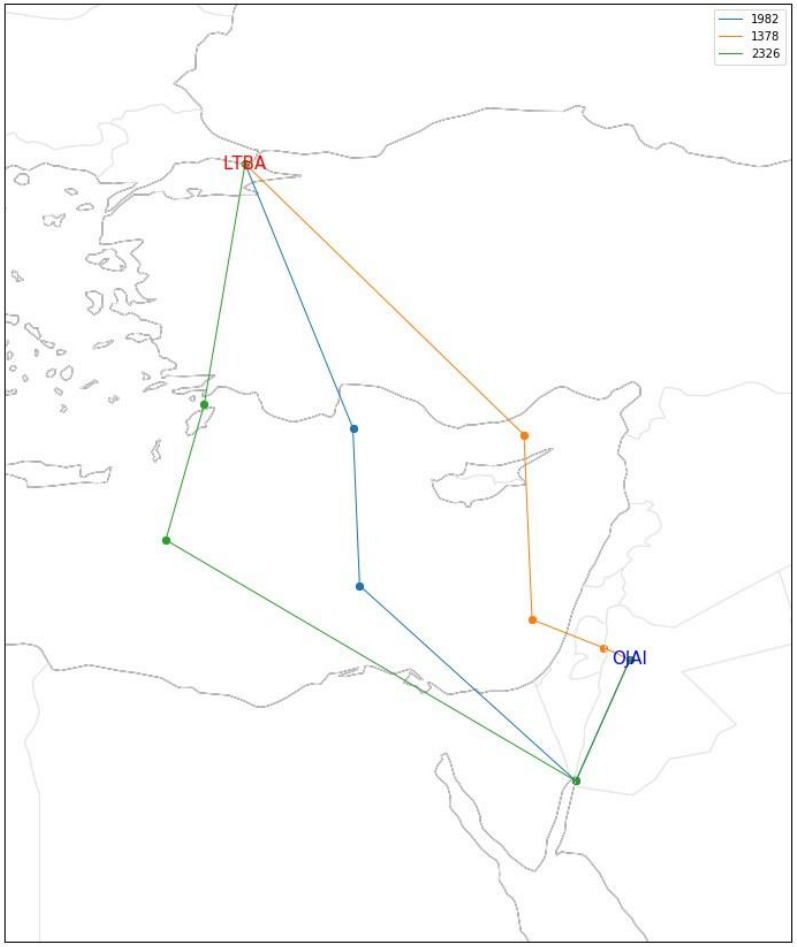
Optimisation

- Look in the literature for guidance.
- Find out the characteristics of the problem:
    - Several variables?
    - Bounded?
    - Discrete optimization? Continuous?
    - One minimum? Several?
    - Computational load of a point estimation?
- Good candidates (scipy.optimize, google_or, or pyomo modules):
    - Basin-hopping, simulated annealing, genetic algorithm: good for unknown energy landscape. Long to converge.
    - Gradient-based methods. Usually good for a unique minimum.
    - (Mixed-)integer linear or quadratic methods. Deterministic.

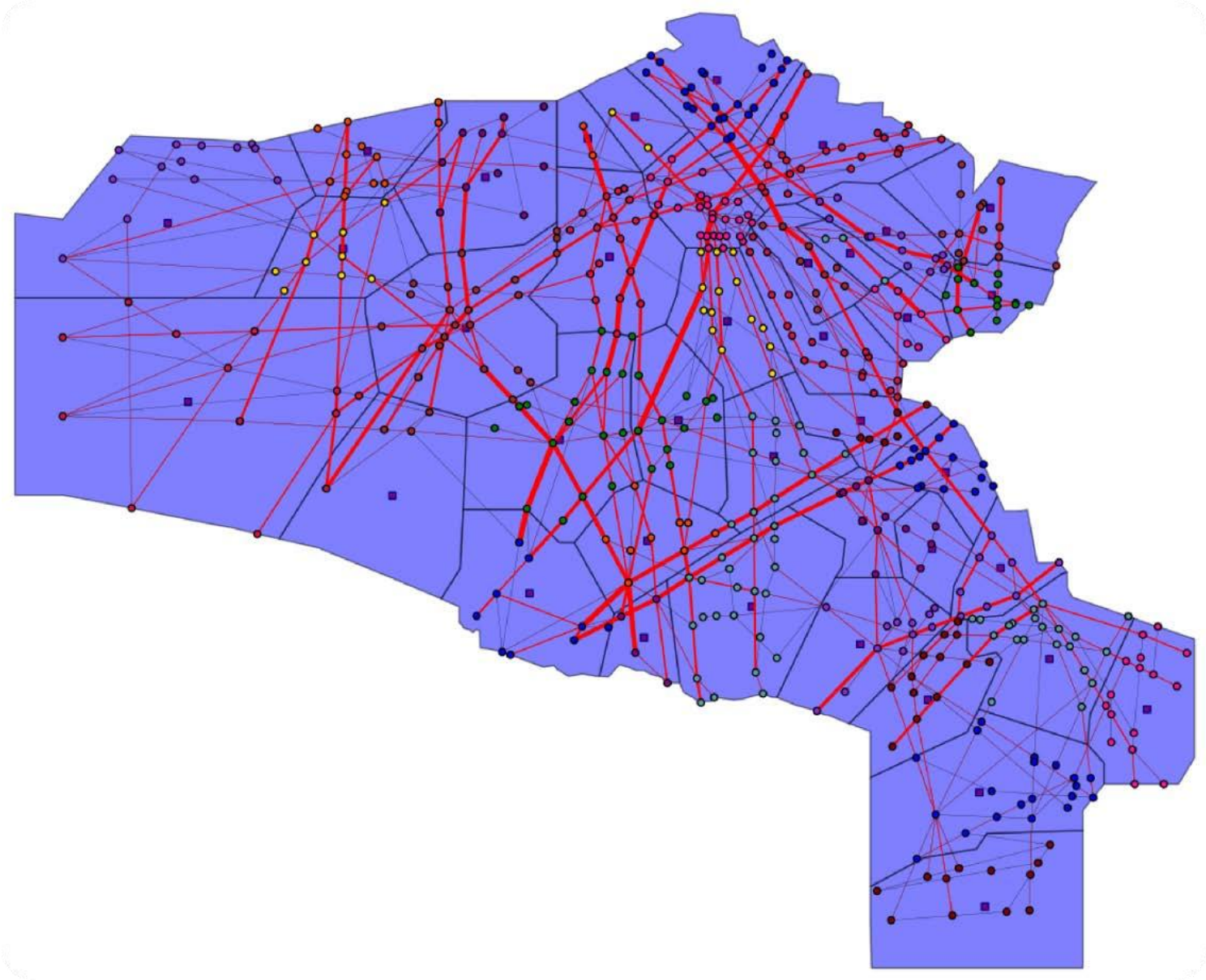
Data visualisation

- For most scientists working with Python, matplotlib is the way to go for graphs. Some improvements:
  - Shapely: easily use polygons. Very useful for airspaces etc.
  - Seaborn: statistical visualisations.
  - Ggplot2: nice plots ☺.
  - Plotly: non-open, API using D3.js under the hood ➔ interactive display. Drawback: has to connect to the server.
- Google map Python (and other languages…) API.
- Blender for 3D visualization (Python console).

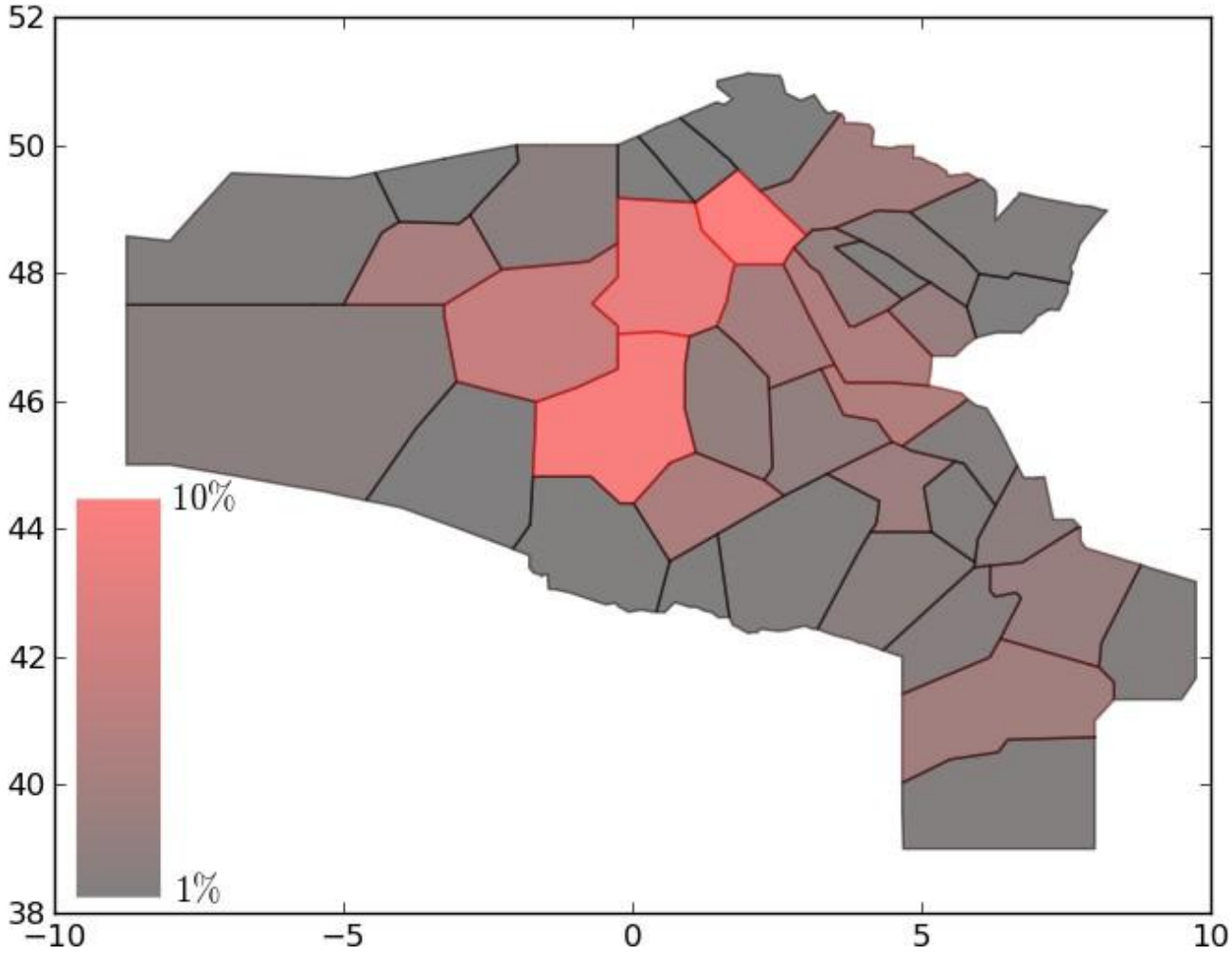
Data visualisation



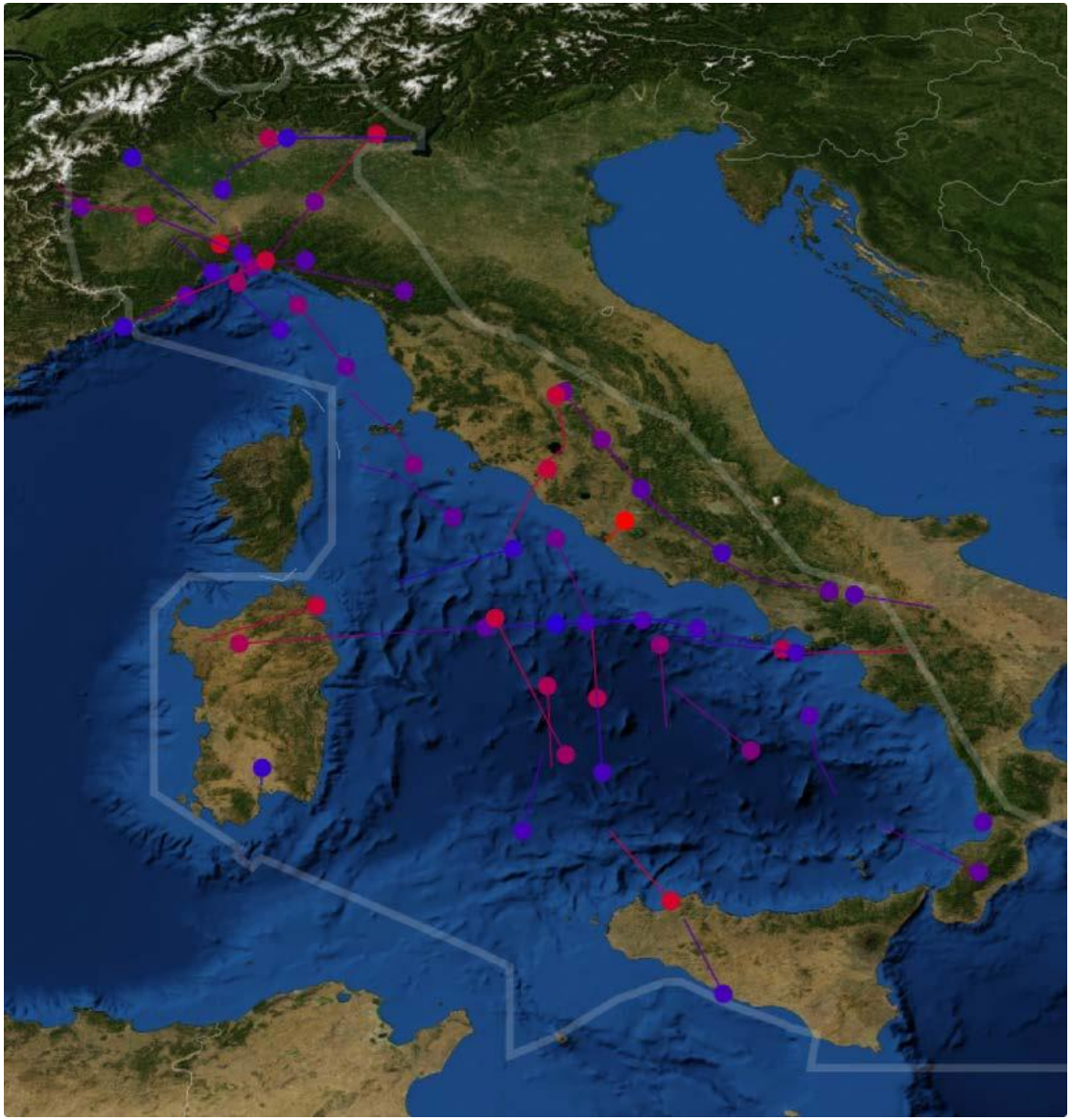
Data visualisation



Data visualisation

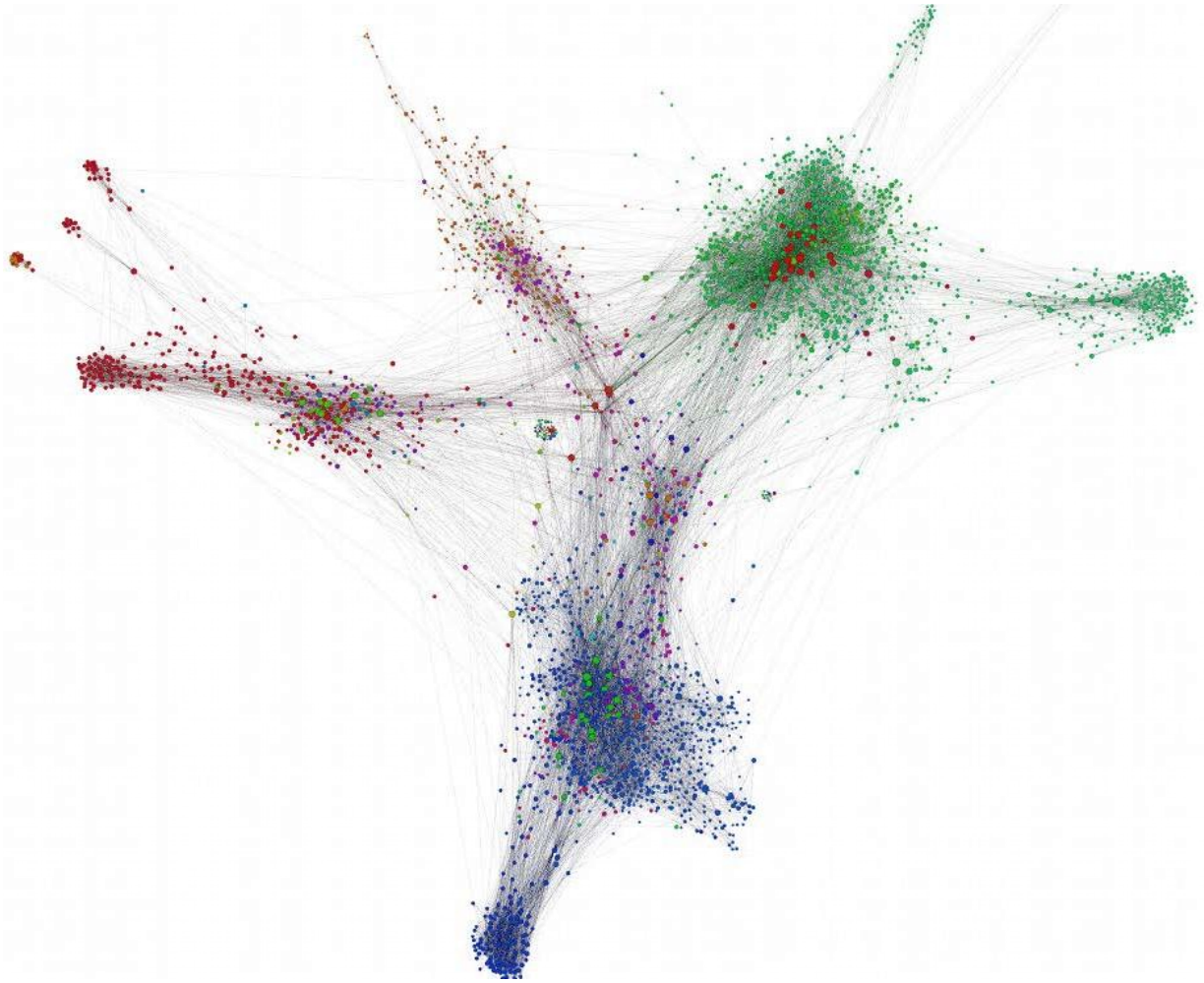


Data visualisation



Data visualisation

- D3.js (free): JavaScript library to do beautiful and interactive graphs.
- Google Earth: accepts XML files to display polygons, trajectories etc.
- Gephi (open source, free) for graphs/networks.



Data visualisation

- FromDady: developed by ENAC.
- Extremely quick, handles massive amount of data.
- Excellent for trajectory data exploration: ‘brush’ ‘pick’, ‘drop’ functions.
- Not open-source.
- Remaining bugs.

